Privacy-Preserving Shopping Intelligence via Federated and On-Device Learning

How much next-product recommendation quality survives when shopping data never leaves the device?

Group members
Ahmed Sherif
Eid Abdelrihem
Ahmed Abdelhamed

Supervised by
Prof. Rongxing Lu, Queen's University
Prof. Mona Farouk, Cairo University
Prof. Motaz El-Saban, Microsoft (industry mentor)

MSc in Artificial Intelligence and Data Science
School of Computing, Queen's University · October 20, 2026



Grant Hall, Queen's University. Photo: Takipoint123, CC BY-SA 4.0, Wikimedia Commons

The research question

“How much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models?”

Why it matters

- A browser sees shopping behaviour on the user's own device
- Cloud training copies that history to a server, which users and regulation resist
- Learning on the device is constrained: little compute, one short history, and updates can leak

What we measure

- **Retention** of cloud quality: federated and on-device systems
- **Server pretraining** (warm start) and **on-device fine-tuning**
- **Privacy**: cost of DP, leakage audits
- **Deployability**: INT8 size, browser latency, upload bytes

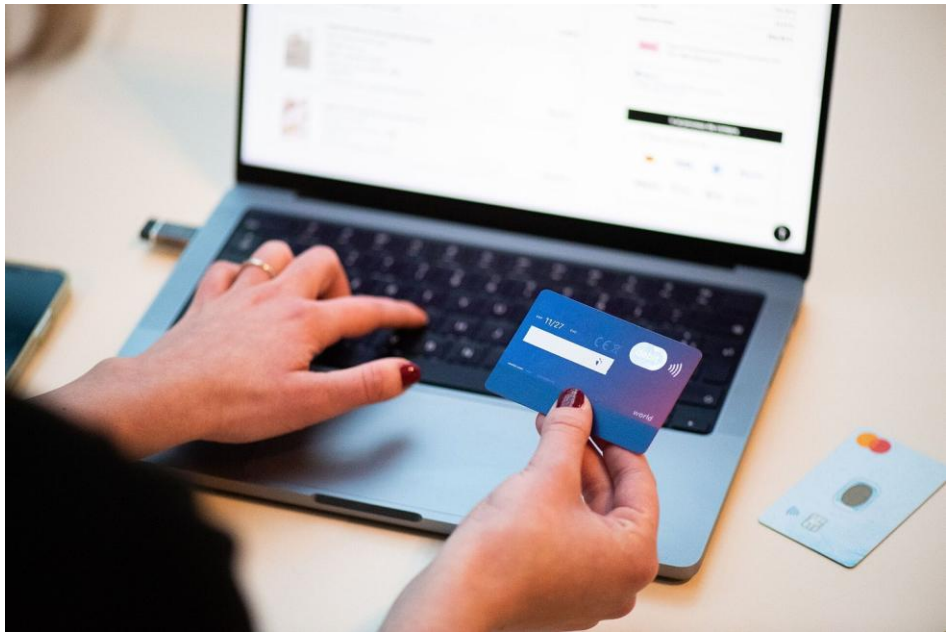
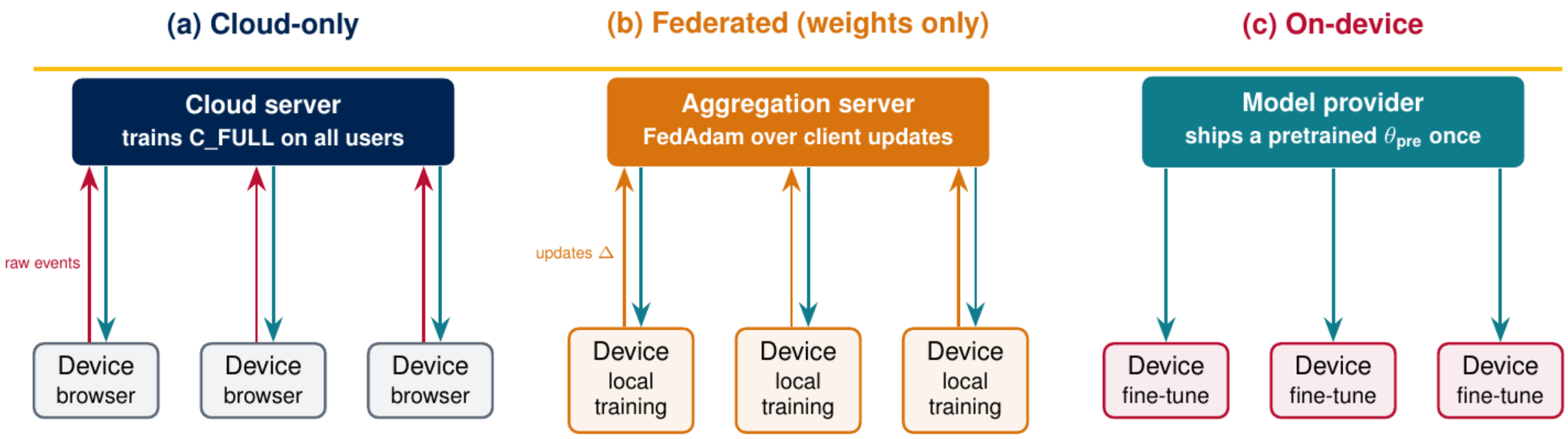


Photo: Bogdan Hoyaux / European Commission, CC BY 4.0, Wikimedia Commons

Three ways to build a shopping recommender

Every result is expressed as retention of the cloud reference



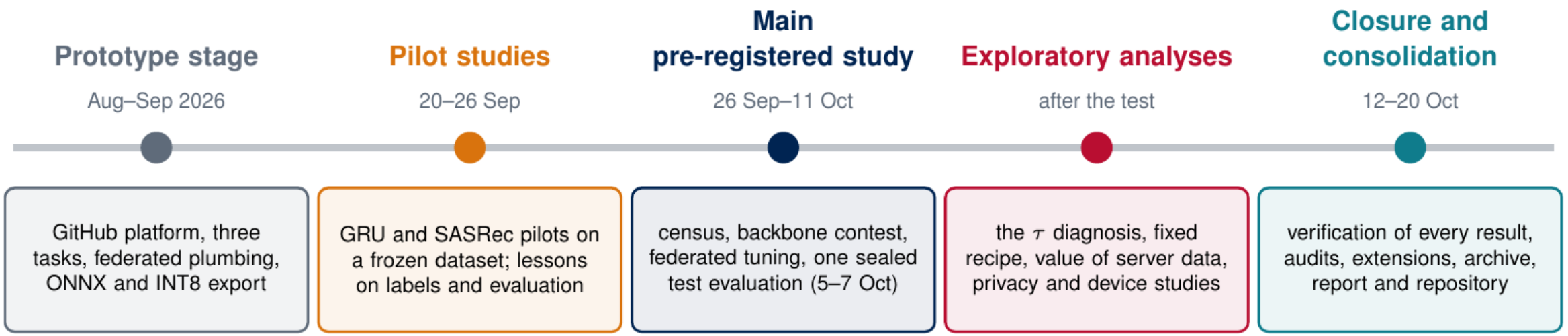
Highest quality, but shopping histories leave the device

Raw events stay local; updates can still leak (needs DP / SecAgg)

Nothing uploaded; quality depends on the shipped model and local data

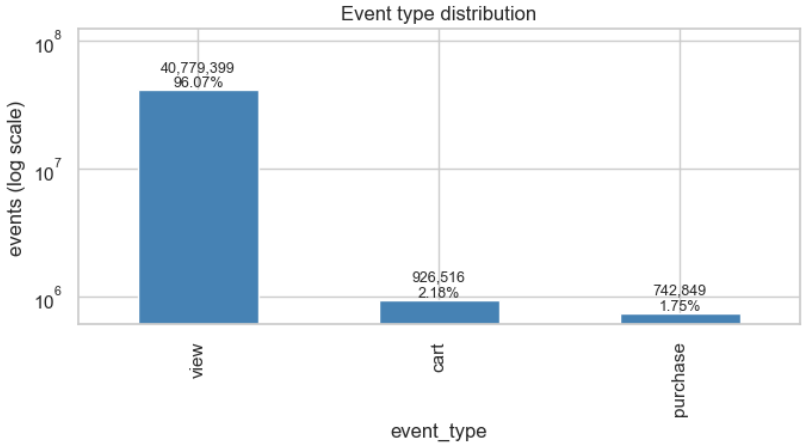
The project at a glance

Each stage answered a question raised by the previous one

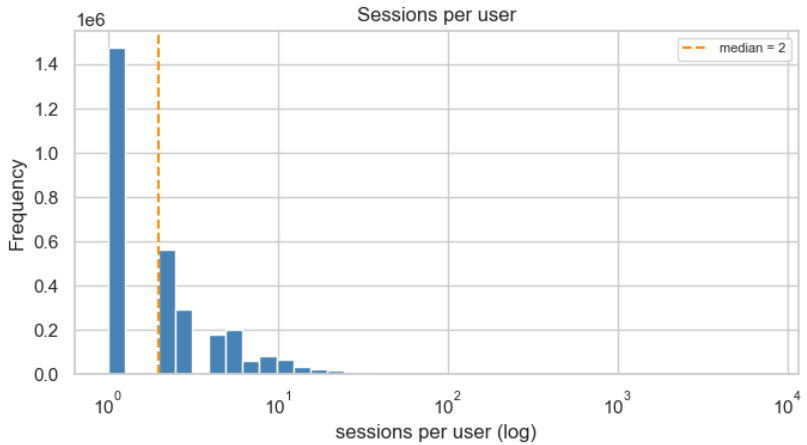


The data: real shopping sessions from one store

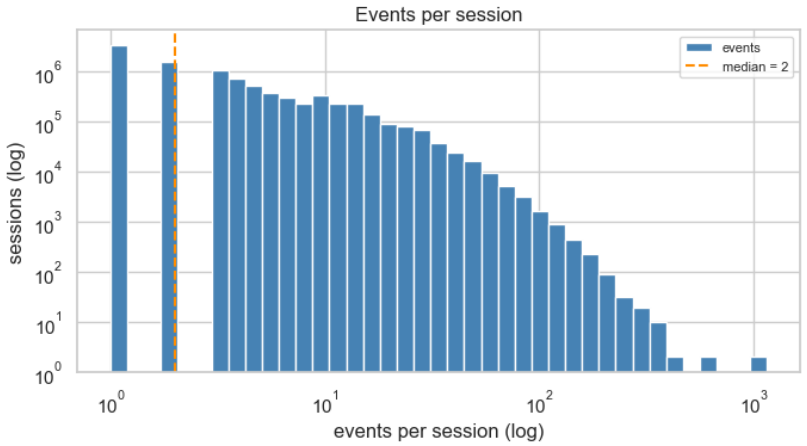
REES46, October 2019, as plotted by our data-audit notebook



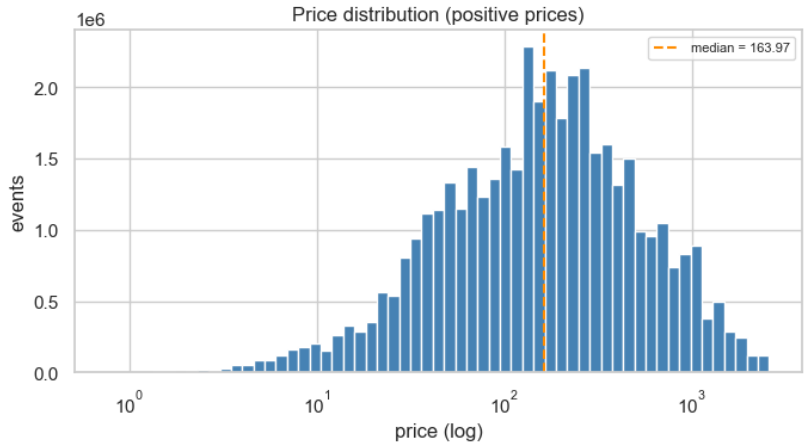
Event types: 96% views, 2.2% carts, 1.8% purchases



Sessions per user: median 2



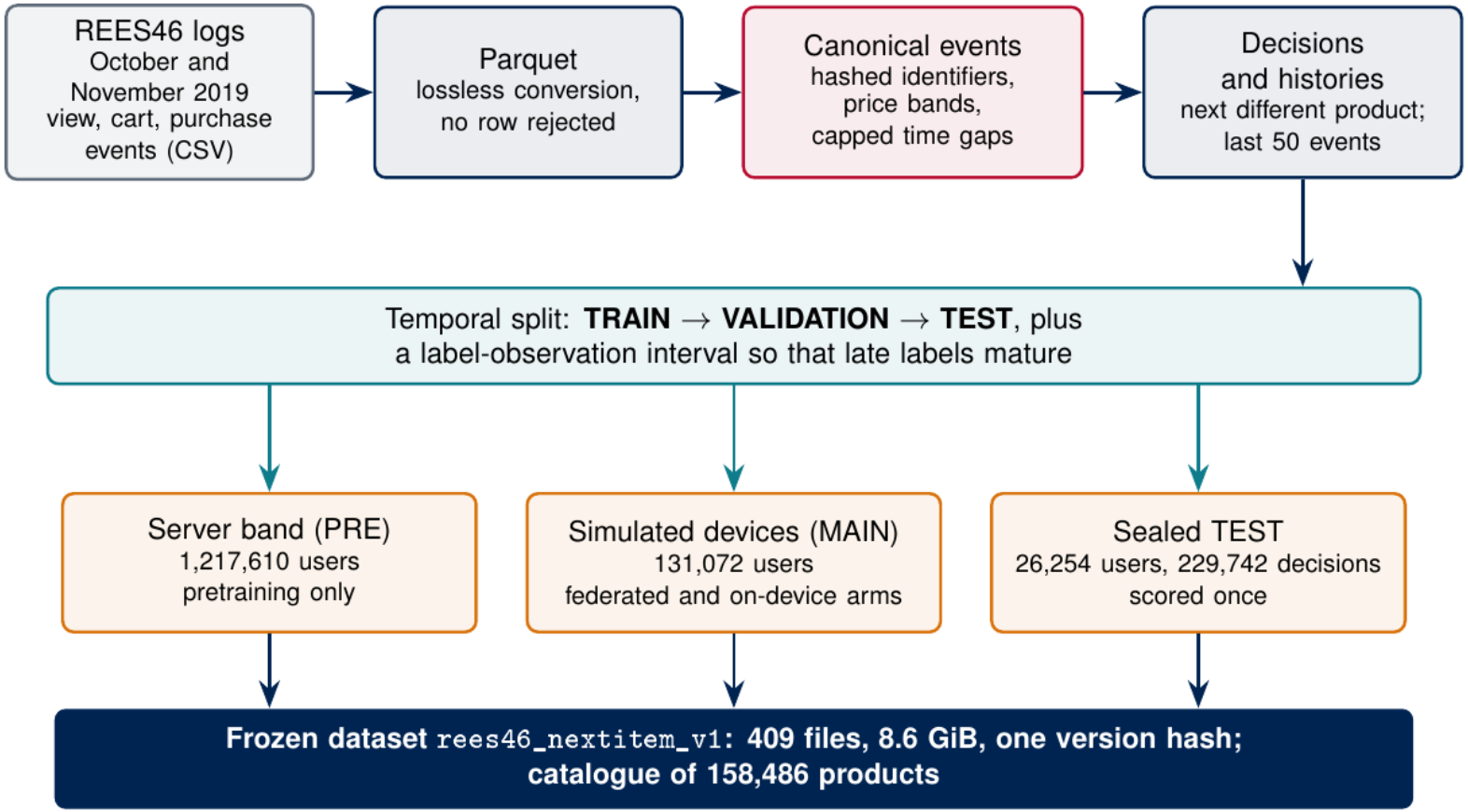
Events per session: median 2, long tail



Product prices: median 163.97

From the raw log to the frozen dataset

Oct–Nov 2019 · full catalogue of 158,486 products · one frozen, hashed dataset

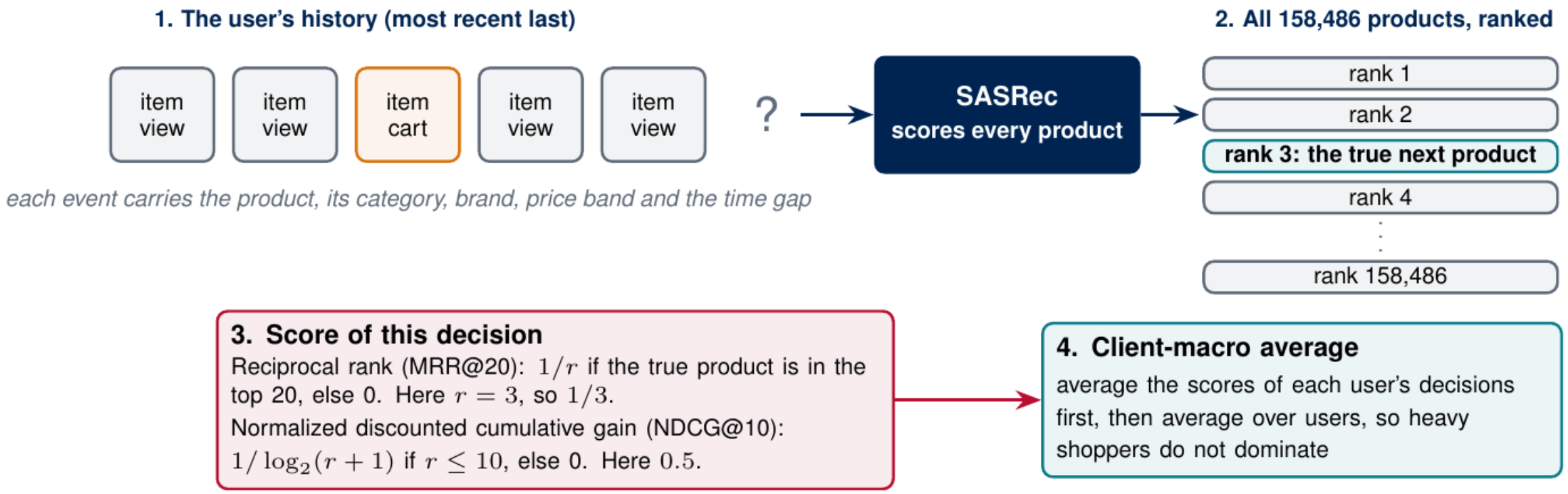


Design choices

- Rank the **full catalogue**, no sampled negatives
- Three disjoint populations: server band, devices, sealed TEST
- SASRec: device d=64 (20.7M) · cloud d=256 (83.9M)
- Every decision locked before **one** TEST look

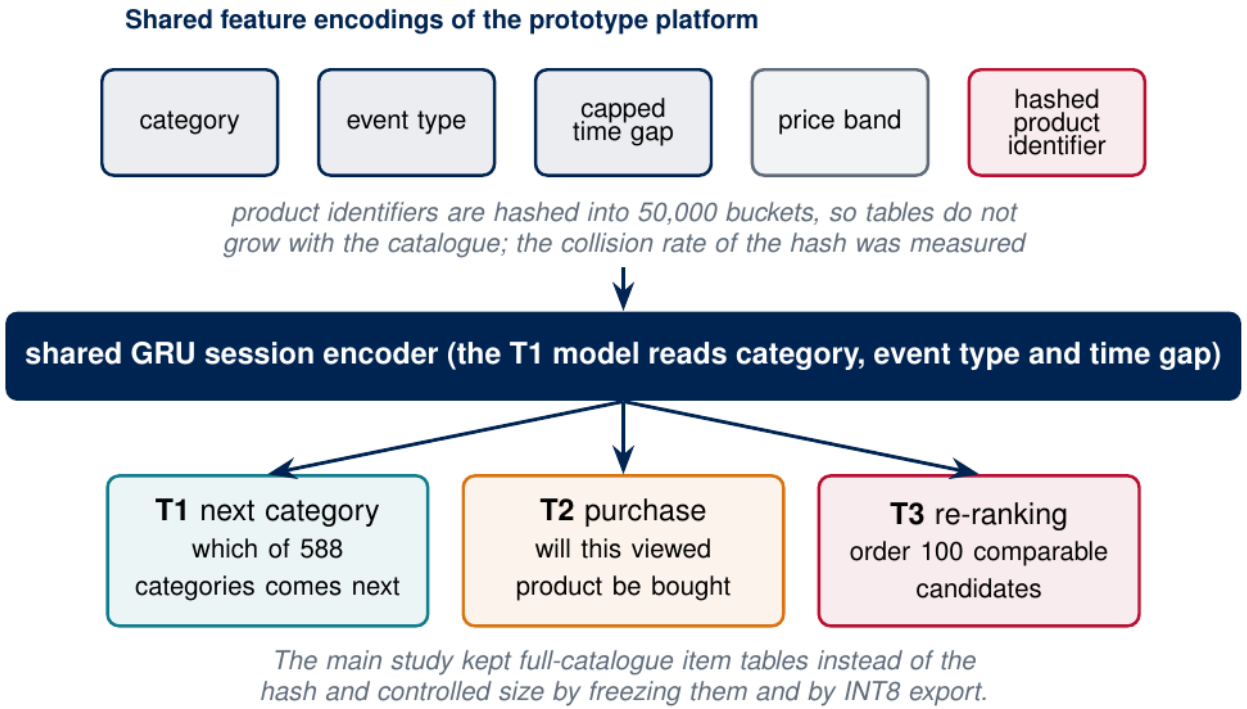
How one recommendation is scored

Mean reciprocal rank (MRR@20), averaged per user, then over users



The prototype platform

August–September 2026, on GitHub

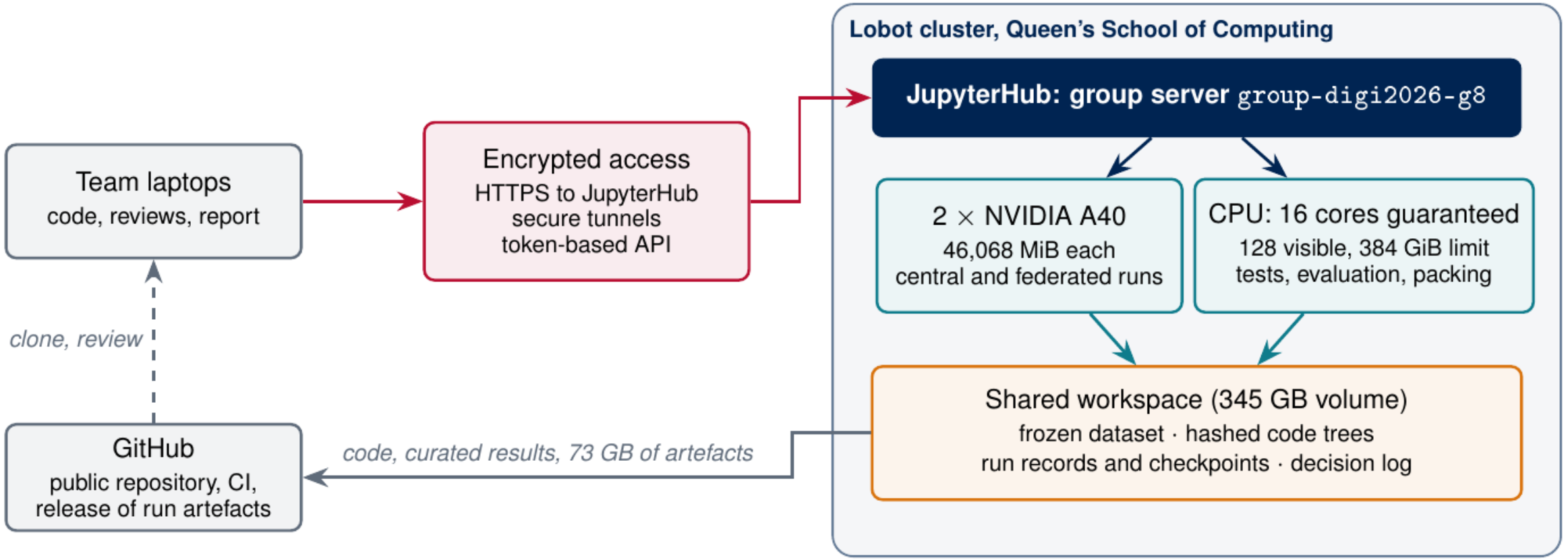


What the prototype delivered

- Data pipeline: 42.4M events → canonical events, sealed TEST window
- Three tasks; GRU beats the transition-table baseline
- Federated plumbing with Flower; pilot retention 88.38% (validation)
- Hashed identifiers, price bands, ONNX and INT8 export, byte ledger, CI
- Lesson: a label defect and evaluator conventions → a stronger protocol

Where the study ran

Queen's Lobot GPU cluster, reached through encrypted access

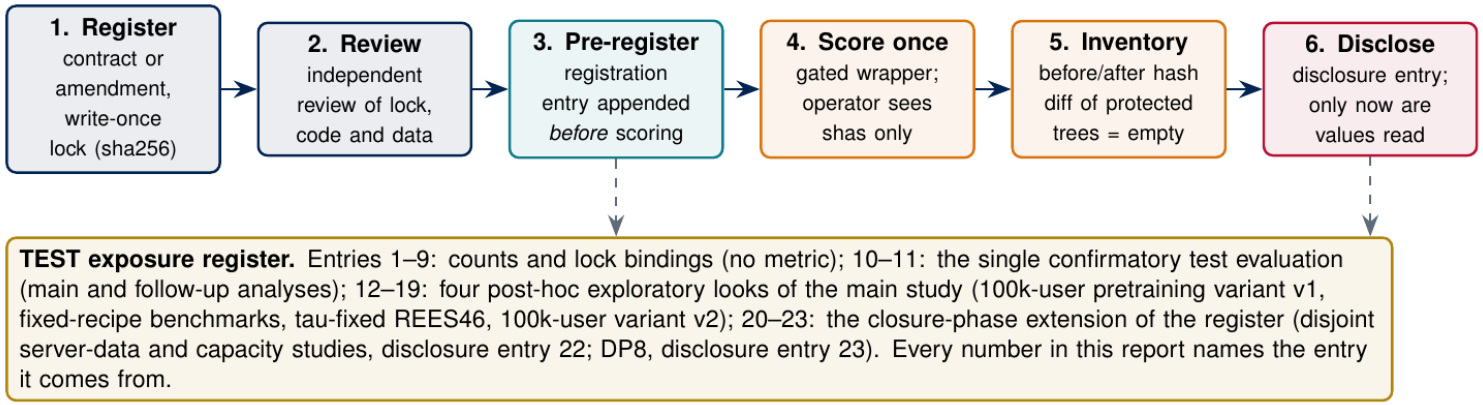


The pre-registered main study

After the pilot studies: a backbone contest, then one sealed test evaluation

Backbone contest (validation)

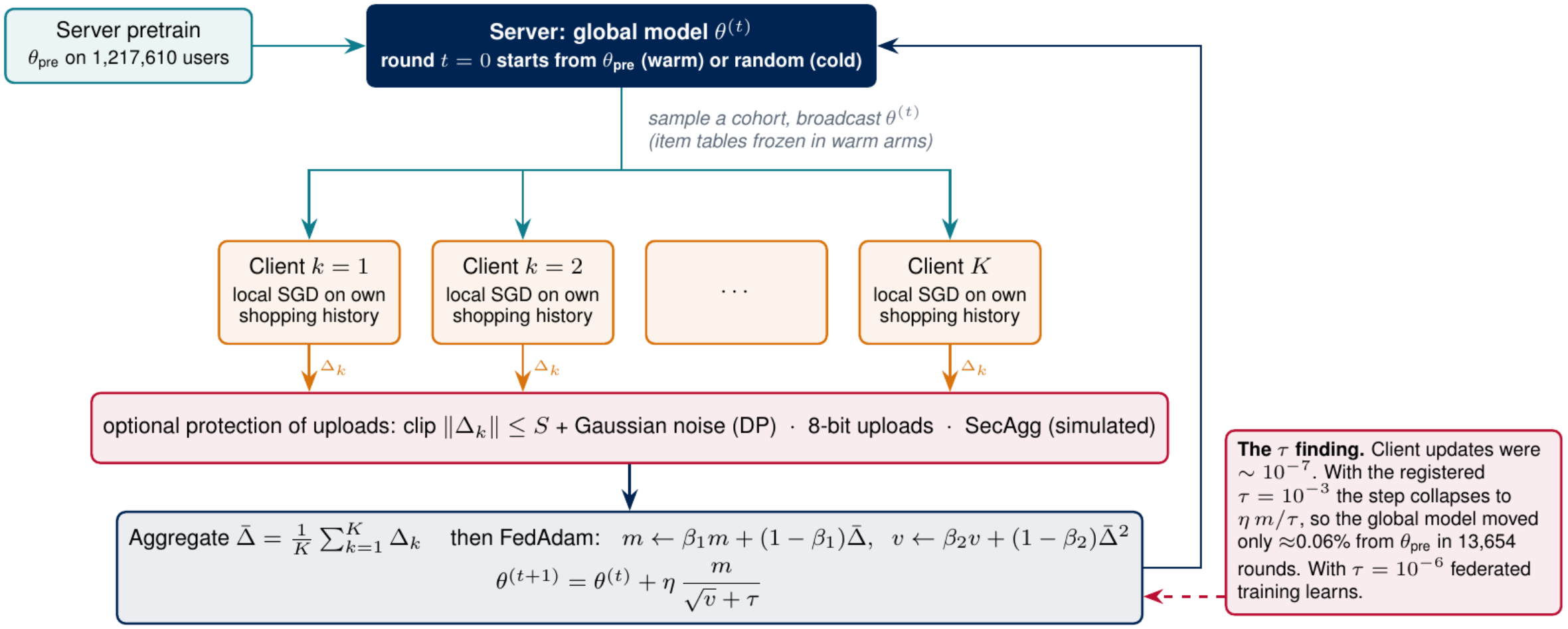
- GRU family mean 0.1569
- SASRec family mean 0.1612
- SASRec locked as the backbone



Why so much protocol?

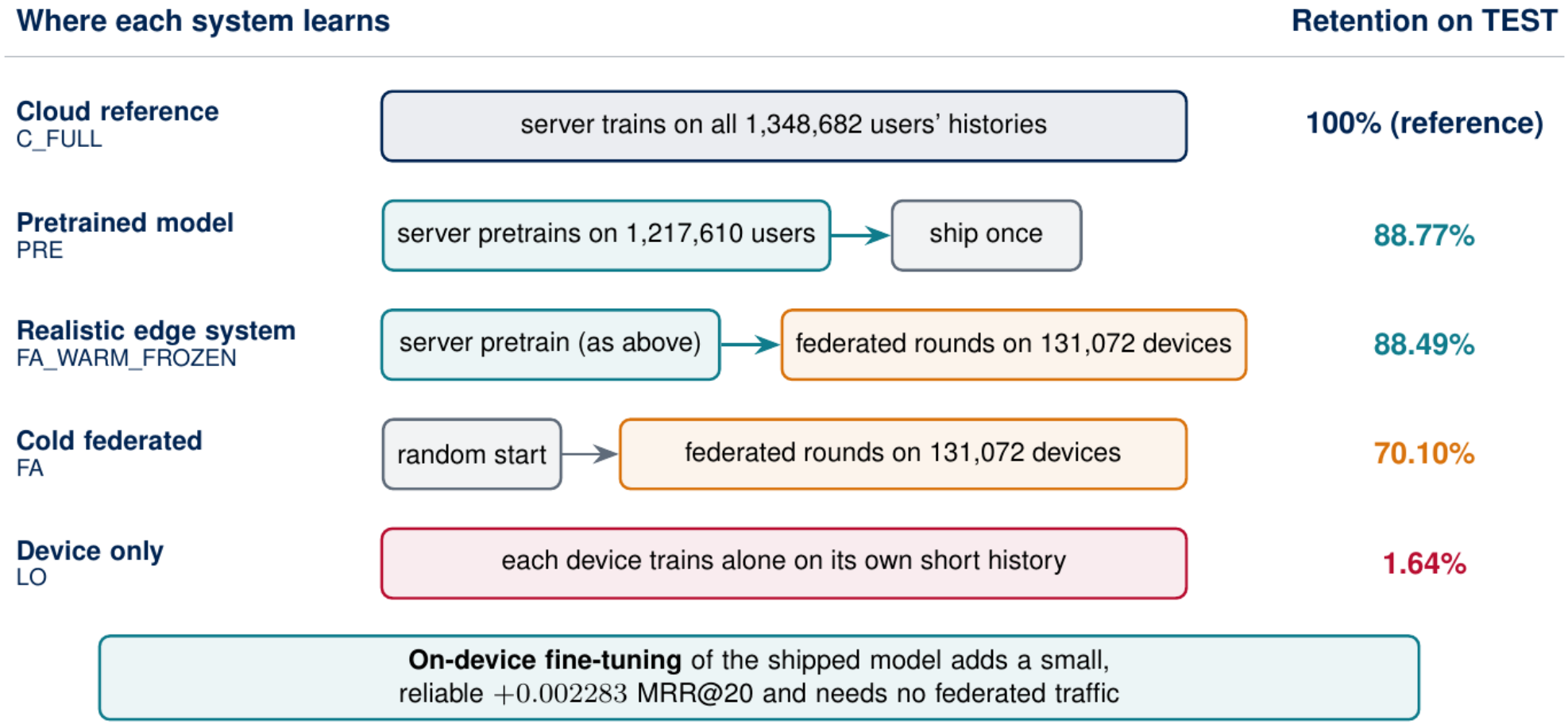
- 33 main-analysis contrasts, two seeds, many arms: easy to fool ourselves
- Selection only on calibration data; validation read once; TEST read once, in a sealed evaluation
- Every later exploratory test look is a numbered, disclosed register entry

The federated recipe: warm-started FedAdam



The confirmatory answer

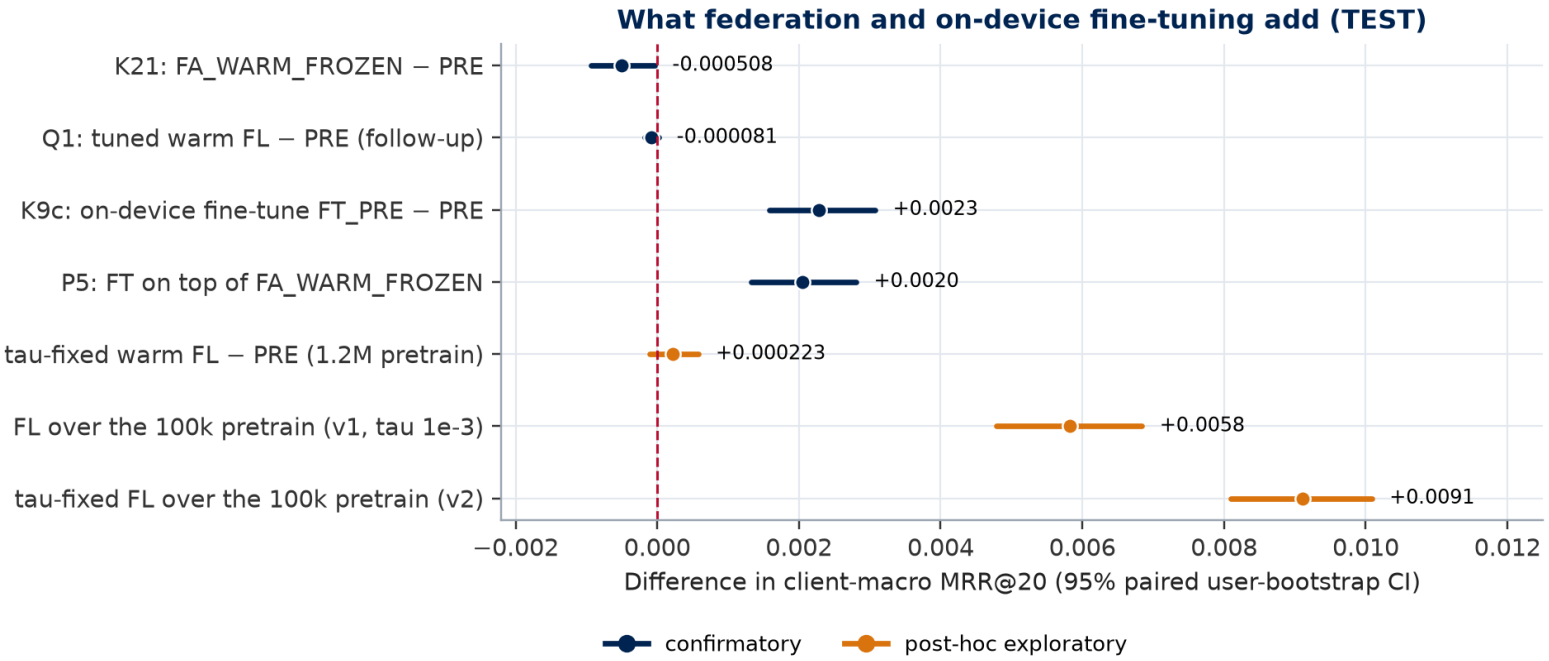
REES46 TEST, 26,254 users, client-macro MRR@20 *(confirmatory result)*



Retention: client-macro MRR@20 of the system as a percentage of the cloud reference, on the sealed TEST population (confirmatory).

What federation and fine-tuning add

Differences in MRR@20 with 95% paired-bootstrap intervals

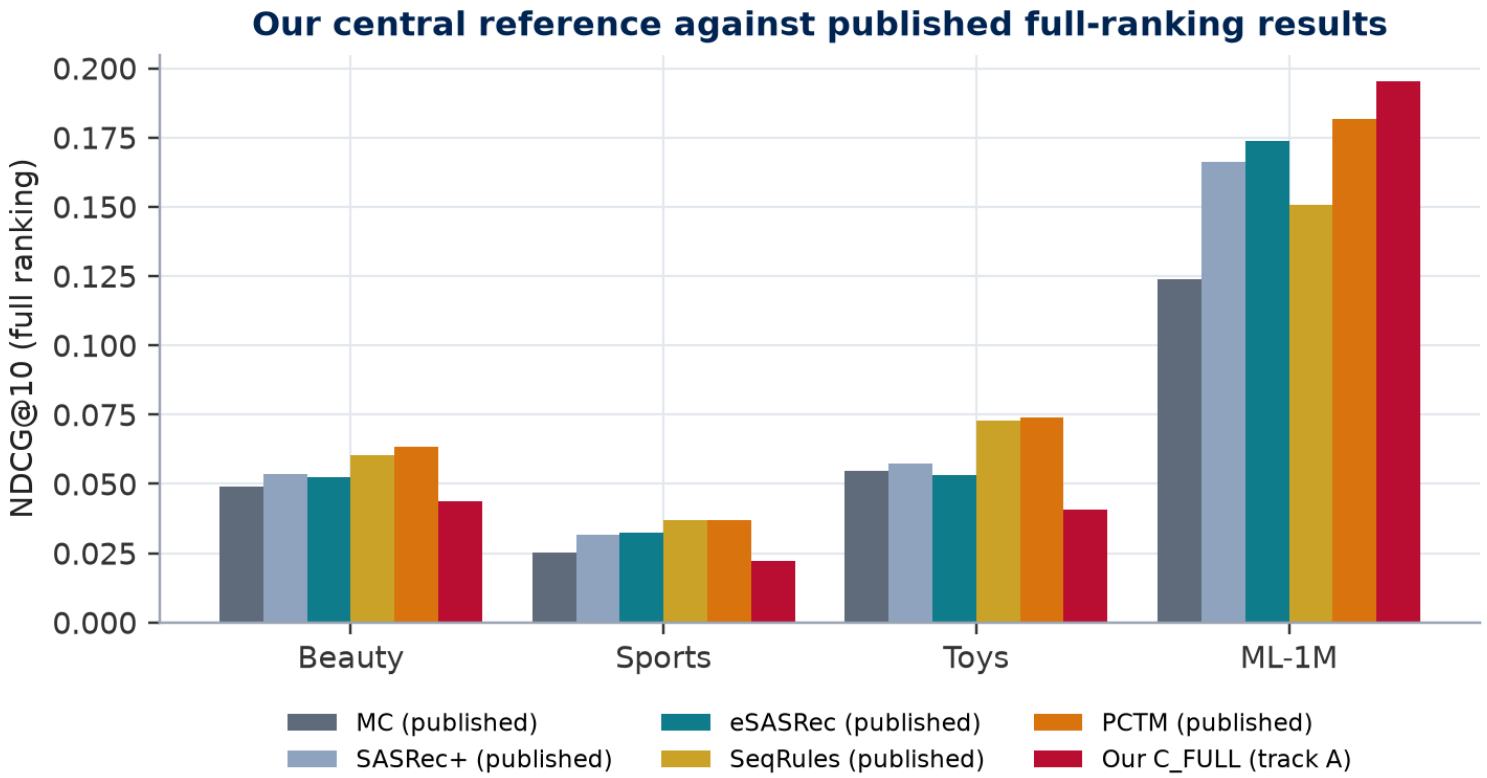


Reading

- **K21** FL – PRE: −0.000508 [−0.000933, −0.000037]: slightly worse than the pretrain alone
- **Q1** tuned warm FL – PRE: equivalent (±0.0005)
- **K9c** on-device fine-tune: +0.002283, the deployable gain
- Orange: post-hoc looks (next slides)

Benchmarks: our cloud reference against published results

Full-catalogue NDCG@10 on public datasets



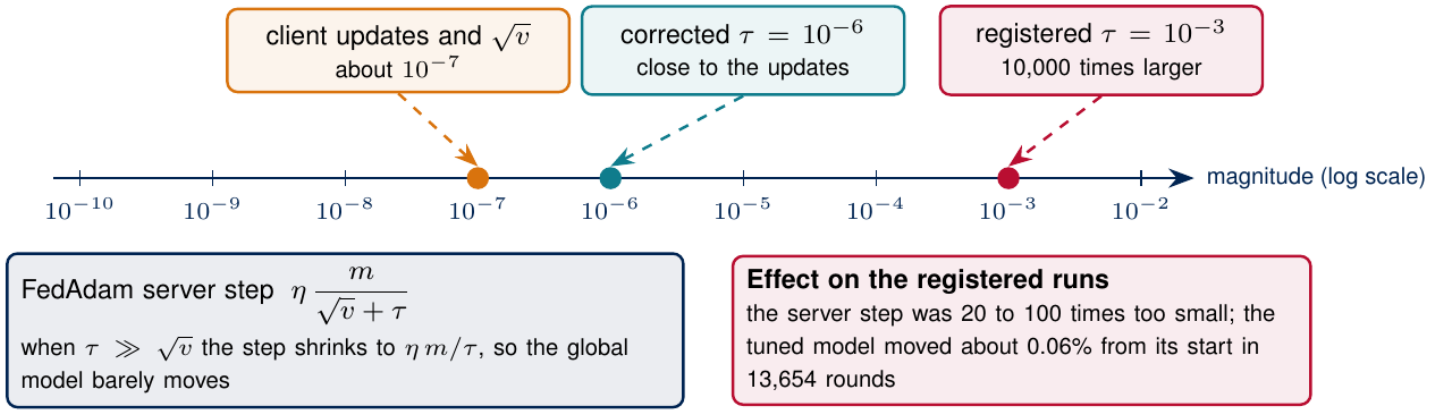
Confirmatory benchmark looks

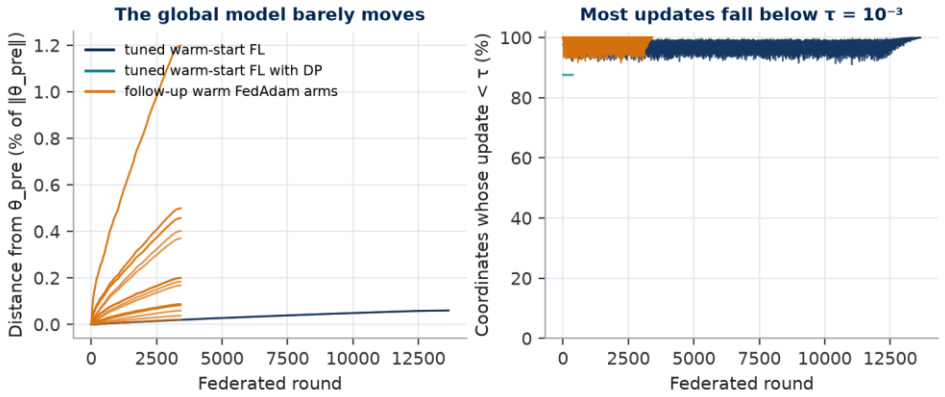
- Our cloud reference is competitive with published full-ranking results
- MovieLens-1M: 0.1936 against 0.1739 published for eSASRec
- Federated arms kept only 13–77% of it
- → something in the federated recipe was wrong

Diagnosis: FedAdam's τ froze federated training

Found after the test evaluation

(exploratory result)





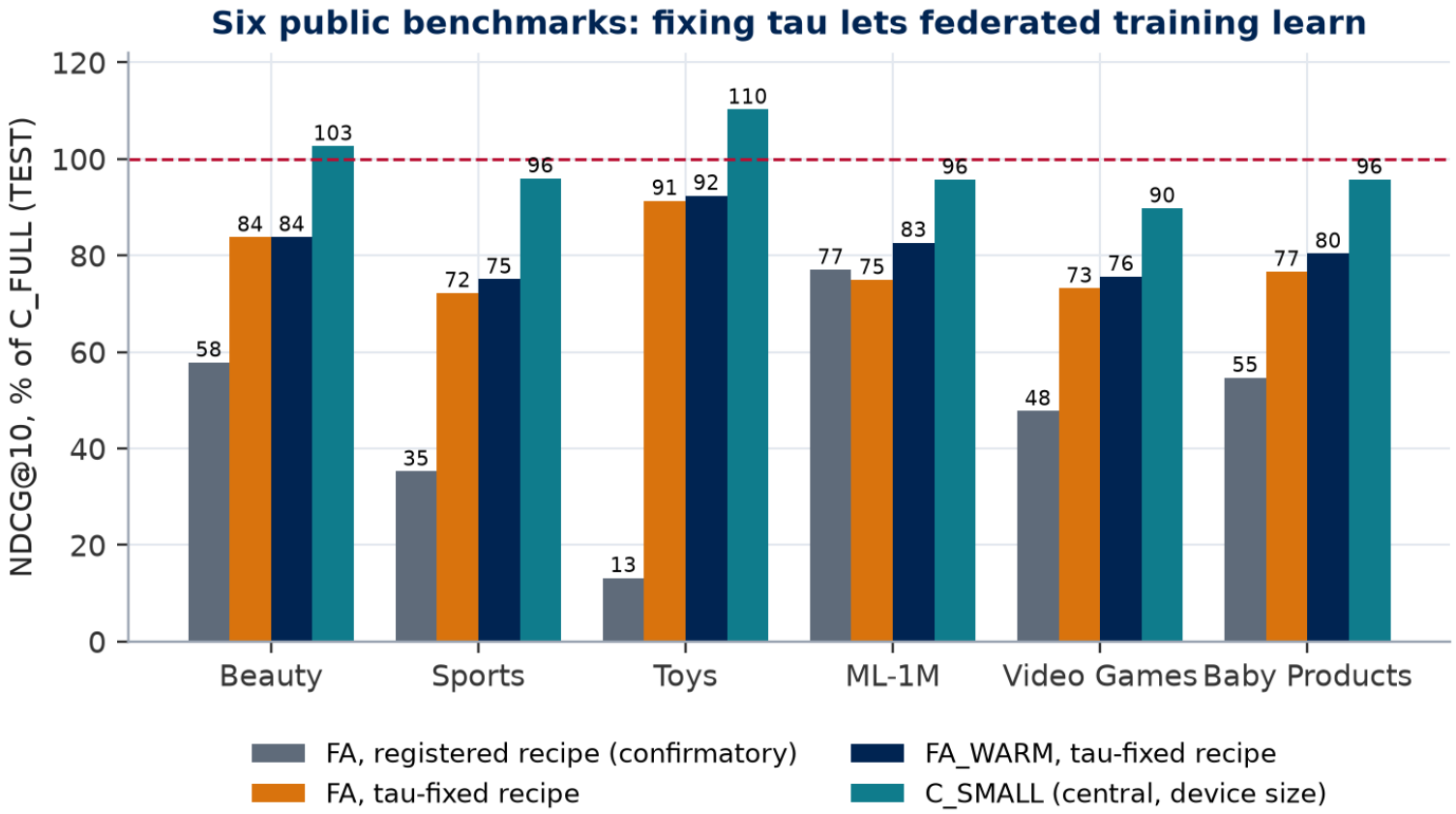
Mechanism

- FedAdam step ∝ m / (√v + τ)
- Updates ≈ 10^{-7}, registered τ = 10^{-3}
- → the server step was 20–100× too small
- The tuned model moved ≈ 0.06% in 13,654 rounds
- Confirmatory results stay as registered; the fix is exploratory

With τ fixed, federated training learns

Six public benchmarks, TEST (exploratory look)

(exploratory result)

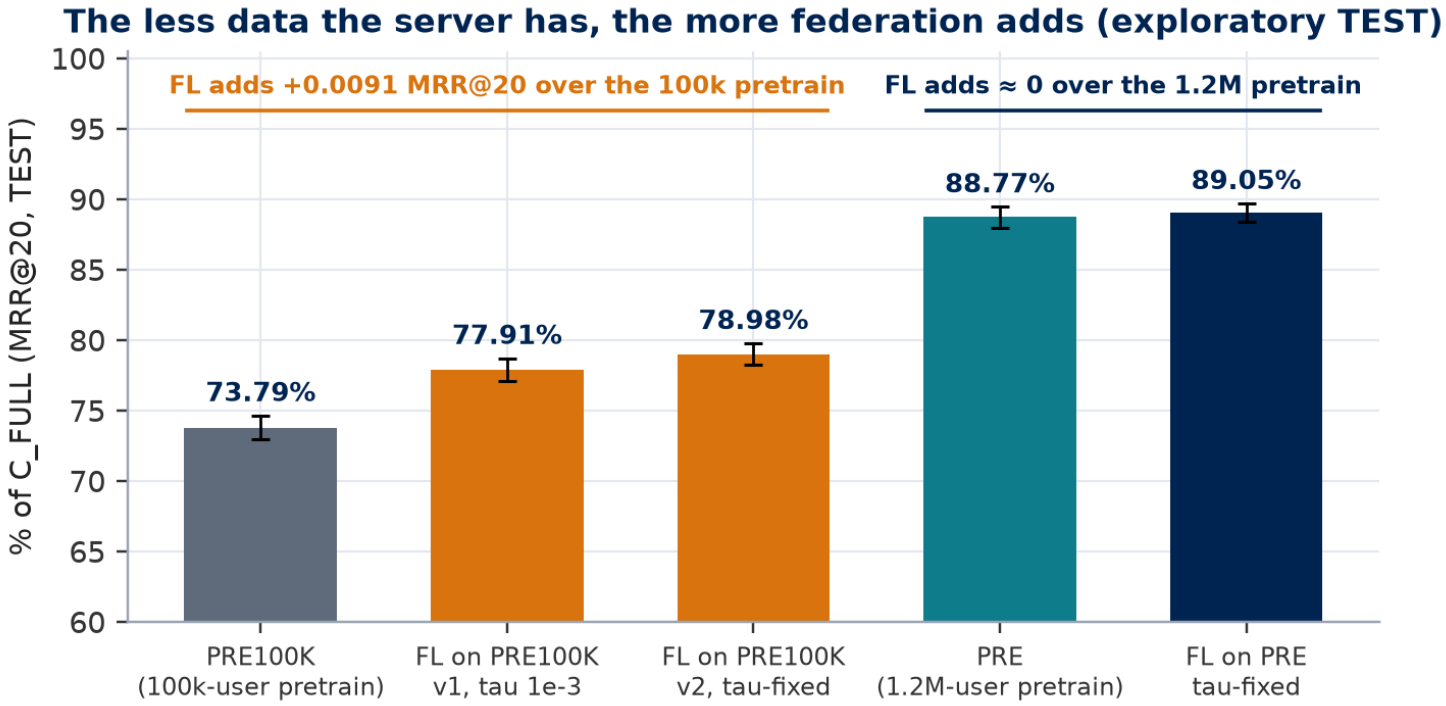


What changed

- Warm FL: 75–92% of the cloud model (was 13–77%)
- Central device-size model 90–110%: size is not the bottleneck
- On REES46 the fix changes nothing (+0.00022, interval includes 0)
- Caveat: τ changed together with the budget and item tables

The value of federation depends on the server's data

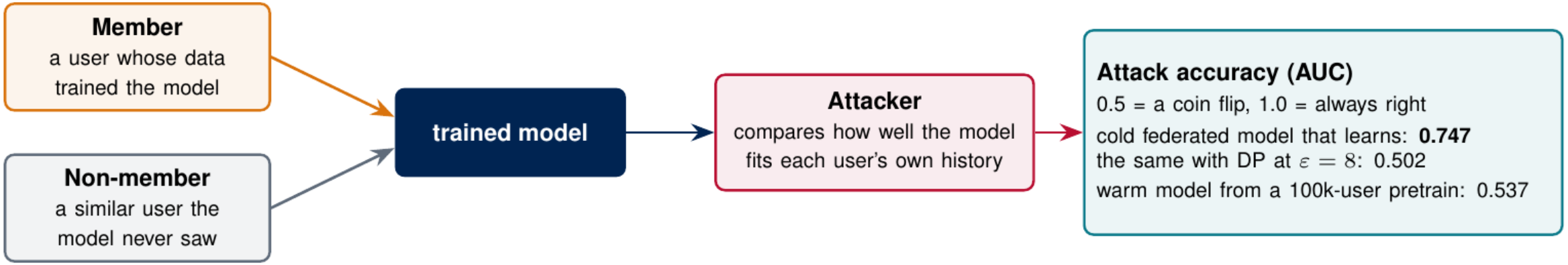
(exploratory result)



Evidence

- 100k-user pretrain: federation adds **+0.0091** MRR@20
- 1.2M-user pretrain: federation adds ≈ 0
- Disjoint server data (50%): warm beats cold by 27–55% on Amazon data
- Exception: MovieLens-1M (6,040 users)

Privacy: learning leaks, and guarantees need care



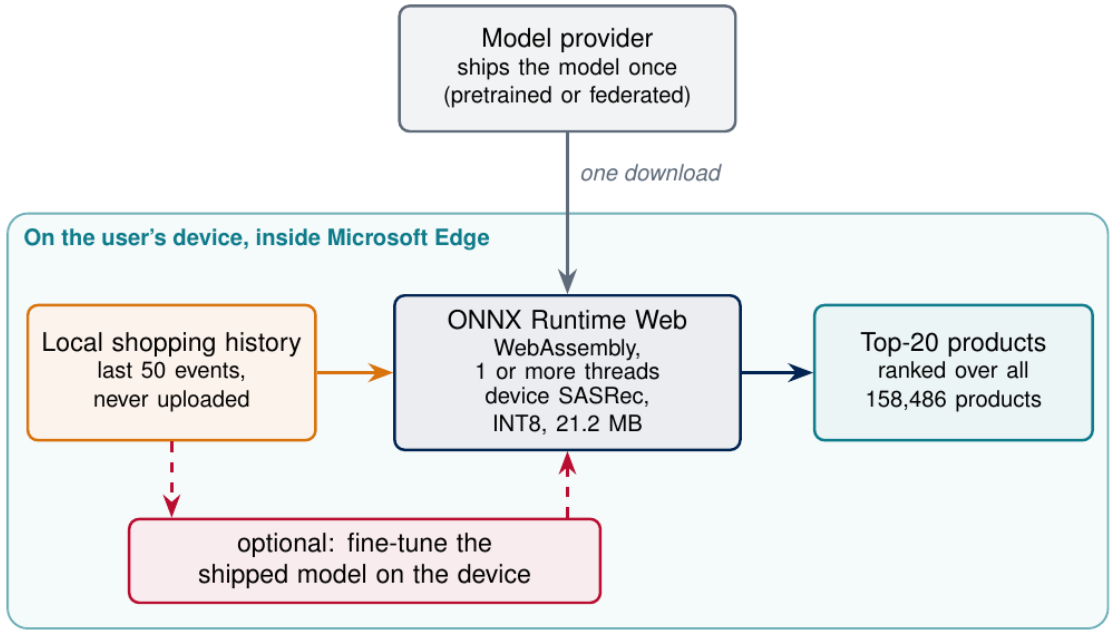
A model that fits its training users much better than unseen users gives membership away; differential privacy limits how much any one user can change the model.

Findings

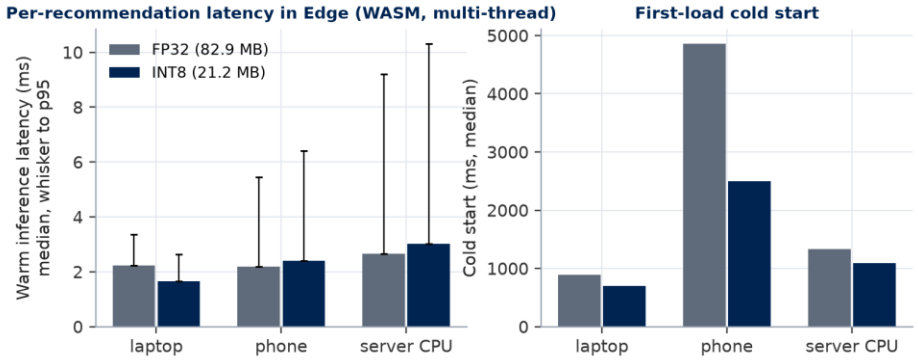
- A federated model that really learns leaks membership (AUC 0.747); DP at ε = 8 removes the measured leak
- DP costs little on the warm system (94.71% kept) and a lot on the cold one (31.30%)
- Audit: the legacy DP randomness gives no formal guarantee; secure aggregation was simulated only

On-device feasibility in Microsoft Edge

ONNX Runtime WebAssembly on a laptop, a phone and a server CPU



warm: 1.7–5.1 ms per query
cold start: 0.7–5.2 s
(median over runs)

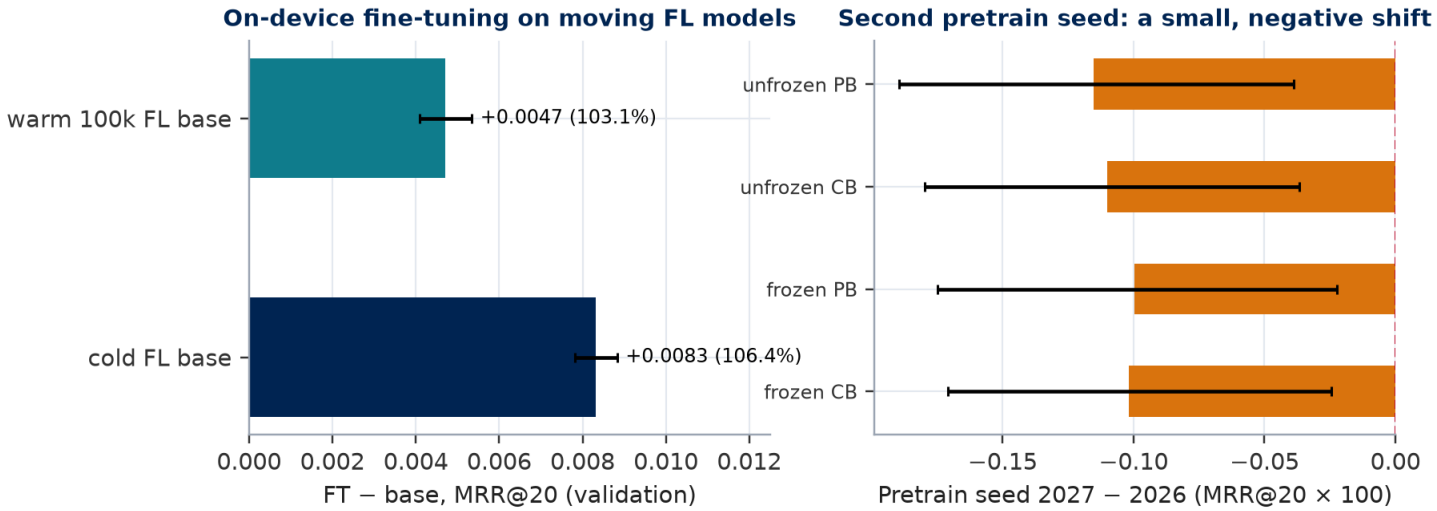


Deployability

- INT8 model: 82.9 MB → 21.2 MB
- INT8 and 8-bit uploads: no measurable quality cost
- Warm median latency 1.7–5.1 ms per query
- Phone cold start ≈ 2.5 s with INT8

Closure experiments

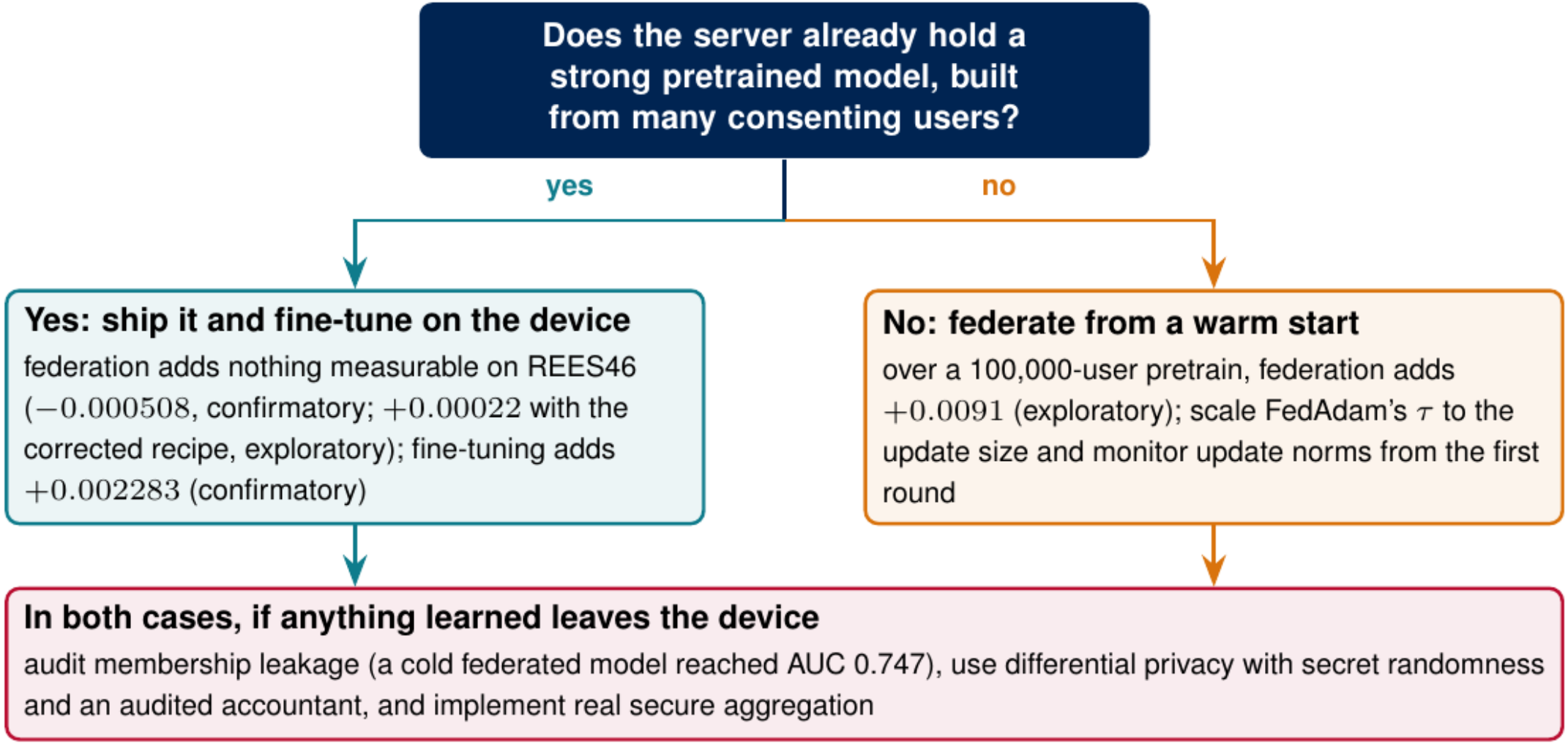
Technical closure and extensions, 12–17 October



What closure added

- Second pretrain seed: small negative shift (−0.10 pts)
- Fine-tuning moving FL models: +3–6%
- Training-free baselines: best Markov chain 0.1366 (validation)
- Stopped or not run: DP-FTRL, canary restart, December replication
- All results labelled; nothing re-scored

What this means for a browser shopping assistant



What we learned

What worked

- Warm start from a server pretrain: the biggest single gain
- On-device fine-tuning: small, significant, cheap
- INT8 and 8-bit uploads: free compression
- Pre-registration kept the main answer honest
- Full-catalogue evaluation: comparable with published work

What did not

- A badly scaled τ froze FL, invisible to the protocol
- Federation over a strong pretrain: no added value
- Device-only training: unusable
- Legacy DP randomness: no formal guarantee
- Some studies stopped for time (DP-FTRL, December replication)

Conclusion and future work

Answer

- A warm-started on-device or federated system keeps **88.49%** of the cloud model
- Whether federation pays off depends on the server-side pretrain and a correctly scaled optimizer
- Fine-tune on the device; federate when the server knows little
- Privacy must be engineered and audited, not assumed

Future work

- Distil a large teacher into the device model
- DP with secret randomness at full scale
- Real secure aggregation
- Replication on December 2019 data
- A field trial in the browser

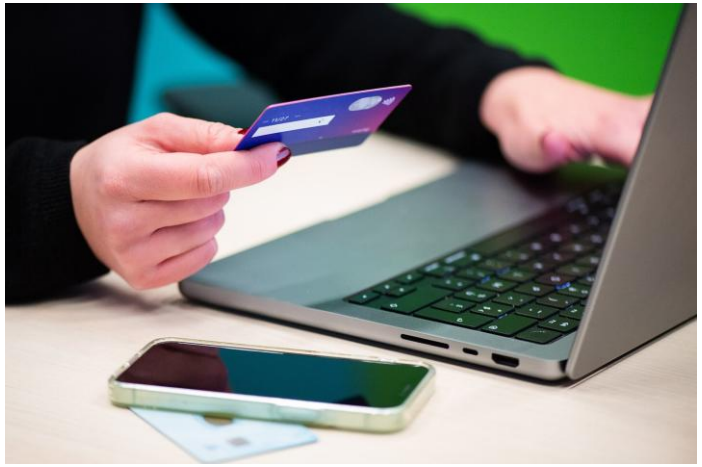


Photo: European Commission, CC BY 4.0, Wikimedia Commons

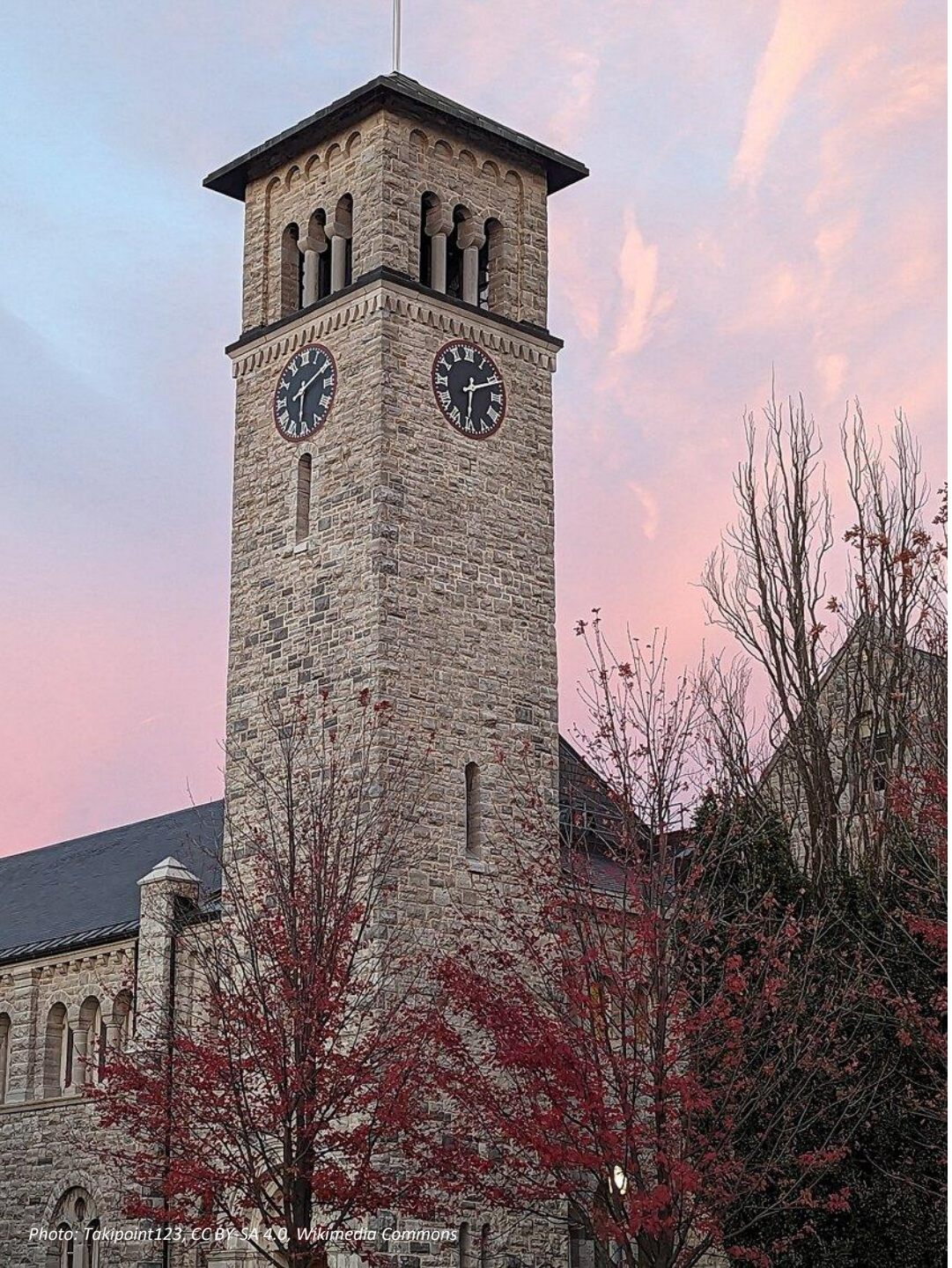


Photo: Takipoint123, CC BY-SA 4.0, Wikimedia Commons

Thank you

Questions and discussion

Code, results and checkpoints:
github.com/microsoftproject26/
Privacy-Preserving-Shopping-Intelligence